

RAG 解决的是：大模型本身没有你的私有知识、最新知识和可验证业务资料的问题。

所以 RAG 的本质不是“让模型变聪明”，而是给模型接上一个**外部知识检索层**。

它主要解决四类问题：

问题	RAG 如何解决
模型不知道私有数据	把私有文档、数据库、API 加载进 RAG 系统
模型知识过时	通过更新知识库接入最新资料
模型容易幻觉	要求模型基于检索到的上下文回答
回答不可追溯	返回 source nodes、页码、文件名、原文片段

2. RAG 和微调的区别

RAG 和微调都可以增强大模型，但它们解决的问题不同。

2.1 RAG 是“外接资料库”

RAG 不改变模型参数。它是在模型外部增加一个检索系统：

用户问题 → 检索知识库 → 找到相关材料 → 材料 + 问题交给 LLM → 生成回答

适合解决：

- 1. 私有知识问答；
- 2. 最新资料问答；
- 3. 需要引用原文依据的问题；
- 4. 知识经常更新的场景；
- 5. 要求可追溯、可审计的业务场景。

2.2 微调是“改变模型行为或能力”

微调会更新模型参数，让模型学会某种风格、格式、任务模式或领域表达方式。

适合解决：

- 1. 固定格式输出；
- 2. 特定语言风格；
- 3. 特定任务范式；
- 4. 复杂分类、抽取、改写；
- 5. 让模型更像某类专家表达。

但微调不适合频繁注入大量事实知识。原因是：

- 1. 成本高；
- 2. 更新慢；
- 3. 很难保证模型准确记住所有细节；
- 4. 无法自然提供来源引用；
- 5. 旧知识容易残留。

2.3 总结

维度	RAG	微调
是否改模型参数	不改	改
主要解决	知识接入、事实更新、引用溯源	行为模式、输出风格、任务能力
适合数据	文档、知识库、数据库、研报	标注样本、问答对、指令数据
更新成本	低，更新知识库即可	高，需要重新训练或增量训练
可追溯性	强，可引用原文	弱，模型内部知识不可直接溯源
金融场景适配	非常适合制度、研报、投研知识库	适合格式化写作、分类、抽取、风格对齐

一句话：

RAG 解决“模型该参考什么资料”；微调解决“模型该怎么表达、怎么完成任务”。

在金融场景里，通常优先用 RAG 接事实和资料，再用 prompt engineering 或轻量微调控制输出格式。

3. Chunk size 为什么重要？

Chunk size 指把长文档切成多大的文本块。LlamaIndex 文档强调，加载数据后需要进行 chunking、提取 metadata、embedding，每个 chunk 才能被检索并更好地供 LLM 使用。

它重要的原因是：**RAG 检索的基本单位通常不是整篇文档，而是 chunk / node。**

LlamaIndex 也说明，Node 是 Document 的原子数据单元，代表源文档中的一个 chunk，并带有元数据。

3.1 Chunk 太大会怎样？

如果 chunk 太大：

1. 检索结果不精准；
2. 一个 chunk 里混入太多无关信息；
3. LLM 上下文被浪费；
4. 成本上升；
5. 答案容易被噪声干扰。

例如一份 30 页研报，如果每个 chunk 是 3000 字，用户问“银行净息差观点”，检索回来的 chunk 里可能同时包含宏观、地产、估值、风险提示，模型很难聚焦。

3.2 Chunk 太小会怎样？

如果 chunk 太小：

1. 上下文断裂；
2. 一个观点的因果链被切开；
3. 表格标题和数据分离；
4. 模型只看到局部句子，容易误解；
5. 引用溯源虽然精细，但回答不完整。

例如：

Chunk A：我们认为银行板块估值修复的核心在于.....Chunk B：净息差企稳和地产风险缓释。

如果只检索到 Chunk A，没有 Chunk B，模型就无法完整回答。

3.3 合理 chunk size 的目标

好的 chunk 应该满足三个条件：

1. **语义完整**：一个 chunk 内最好包含一个相对完整观点；
2. **检索精准**：不要混入太多无关主题；
3. **上下文可用**：放进 LLM 后能支持回答。

所以 chunk size 本质上是在平衡：

语义完整性 ↔ 检索精准度 ↔ 成本与上下文长度

LlamaIndex 示例中使用了 `SentenceSplitter(chunk_size=512, chunk_overlap=10)`，说明框架支持通过 splitter 自定义 chunk size 和 overlap。

在研报场景，一般会采用：

正文观点：500–1000 中文字表格说明：按表格或小节整体切风险提示：单独切摘要/投资要点：按 bullet 或段落切 chunk_overlap: 50–150 字

4. Embedding 检索和关键词检索各自优劣

4.1 Embedding 检索是什么？

Embedding 检索是把文本转换成向量，再根据语义相似度查找相关内容。LlamaIndex 文档说明，embedding 是文本语义的数值表示；两个语义相近的文本，即使字面不同，也会有数学上相似的 embedding。

4.2 关键词检索是什么？

关键词检索通常基于 BM25、倒排索引或关键词匹配。

例如用户搜索：

“净息差”

系统会优先找包含“净息差”这个词的文本。

4.3 对比

维度	Embedding 检索	关键词检索
核心逻辑	查语义相似	查词面匹配
优点	能理解同义表达、近义表达、语义相关	精准、可解释、对专有名词友好
缺点	可能召回语义相关但事实不相关的内容	召回受关键词限制，容易漏掉同义表达
适合问题	概念性、自然语言、开放问题	公司名、指标名、法规条款、代码、术语
金融场景风险	可能混淆相似行业/相似概念	用户换一种说法可能搜不到
可解释性	相对弱，需要看相似度和原文	强，命中词清晰

4.4 金融场景更适合混合检索

在研报系统中，不会只用 embedding，也不会只用关键词，而是用 **Hybrid Retrieval**：

Embedding 检索：解决语义召回关键词 / BM25：解决专有名词、指标、公司名、代码精准召回
Metadata Filter：解决年份、行业、机构、报告类型过滤Rerank：对候选结果重排

比如用户问：

“宁德时代的毛利率压力主要来自哪里？”

需要：

- 1. 关键词锁定“宁德时代”；
- 2. embedding 找到“盈利能力承压”“价格竞争”“原材料价格”等语义相关段落；
- 3. metadata 限定报告日期和行业；
- 4. reranker 判断哪些段落真正回答问题。

5. Rerank 为什么重要？

Rerank 是对初步检索结果进行重新排序。

RAG 中第一轮 retrieval 通常是“召回”，目标是尽量不要漏掉相关材料。但召回结果里往往有噪声，排序也未必最符合用户问题。Rerank 的作用是：**在候选结果中重新判断谁最相关、谁最应该放进上下文。**

LlamaIndex 文档中虽然没有直接展开 rerank 的工程细节，但它明确提到 Querying 包括 Postprocessing，Postprocessing 会对检索到的 Nodes 做 rerank、transform 或 filter。

5.1 为什么第一轮检索不够？

因为 embedding 相似度不等于“答案相关性”。

例如用户问：

“这篇研报对银行股的核心风险判断是什么？”

第一轮 embedding 可能召回：

1. 银行板块投资建议；
2. 银行估值修复；
3. 银行风险提示；
4. 银行净息差；
5. 宏观经济风险。

这些都“语义相关”，但真正回答“核心风险判断”的可能是第 3 条。Rerank 的作用就是把第 3 条排到前面。

5.2 Rerank 的价值

Rerank 重要，是因为它能提升：

1. **Top-k 精度**：让最有用的内容排在前面；
2. **上下文质量**：减少无关 chunk 进入 LLM；
3. **回答准确性**：降低模型被噪声误导的概率；
4. **成本效率**：少塞无关文本，减少 token；
5. **复杂问题效果**：尤其适合多跳问题、比较问题、原因问题。

在金融研报场景，rerank 尤其重要。因为研报文本里大量段落语义接近，但问题真正需要的是某一类信息，比如：

- 投资评级；

- 盈利预测；
- 估值假设；
- 风险提示；
- 行业比较；
- 公司核心变化；
- 数据口径说明。

第一轮检索未必能准确分辨这些细粒度意图。

6. 如何做引用溯源？

引用溯源的目标是：用户不仅看到答案，还能看到答案来自哪份文档、哪一页、哪一段。

LlamaIndex 中 Node 本身带有 metadata，可以记录它和源 Document 的关系。文档说明，Node 有与所在 document 和其他 nodes 相关的 metadata。

因此，引用溯源要在 ingestion 阶段就设计好。

6.1 存哪些 metadata？

每个 Node 至少应该保存：

```
doc_id
file_name
report_title
author / institution
publish_date
page_number
section_title
chunk_id
start_char / end_char
source_url 或 file_path
原文片段
```

金融研报场景建议额外保存：

```
stock_code
company_name
industry
```

```
rating
target_price
report_type
data_period
analyst
broker
permission_level
```

6.2 查询时如何返回引用？

查询流程可以设计为：

```
用户问题
↓
Retriever 找到相关 nodes
↓
Postprocessor / reranker 精筛
↓
LLM 基于 nodes 生成答案
↓
答案中插入引用标记
↓
前端展示来源：报告名、机构、日期、页码、原文片段
```

6.3 防止“答案和引用不一致”

一个常见风险是：模型生成的答案看起来有引用，但引用原文并不支持答案。

可以通过几种方式控制：

1. 要求 LLM 只基于 retrieved nodes 回答；
2. 每个关键结论后绑定 node_id；
3. 输出结构化 JSON，例如：

```
{
  "answer": "...",
  "claims": [
    {
      "claim": "银行净息差压力边际缓和",
      "source_node_ids": ["node_123", "node_456"]
    }
  ]
}
```



```
}  
]  
}
```

4. 对每个 claim 做 citation check;
5. 无来源的结论标记为“未在资料中找到”。

LlamaIndex 文档提到 `no_text` 模式可以只运行 retriever，不发送给 LLM，并通过 `response.source_nodes` 检查本来会传给 LLM 的节点。这在调试引用溯源时非常有用。

7. 金融场景下 RAG 的风险点

金融场景的 RAG 风险明显高于普通知识库，因为它涉及事实准确性、合规边界、投资判断和时效性。

7.1 事实幻觉风险

即使有 RAG，模型仍可能：

1. 把不同研报观点混在一起；
2. 把历史观点当成当前观点；
3. 把行业观点套到个股；
4. 编造不存在的目标价、评级或财务指标。

控制方法：

强制引用原文
无来源不回答
关键数据从结构化数据库取，不从模型生成
结论和来源逐条绑定

7.2 时效性风险

金融资料有强时间属性。

同一家公司在不同时间可能：

- 评级变化；
- 盈利预测变化；

- 目标价变化；
- 行业景气度变化；
- 风险判断变化。

如果没有时间过滤，系统可能用旧研报回答新问题。

控制方法：

metadata 必须包含 publish_date
默认优先最新资料
回答中显示资料日期
用户问“当前”时限制时间窗口
过期观点必须提示

7.3 口径混淆风险

金融指标口径复杂，例如：

- 营收 vs 主营收入；
- 归母净利润 vs 扣非归母净利润；
- 静态 PE vs 动态 PE；
- AUM vs AUA；
- 境内收入 vs 境外收入；
- 合并口径 vs 母公司口径。

RAG 如果只做文本检索，很容易把不同口径的数字混用。

控制方法：

指标类问题优先查结构化数据
保留 data_period、unit、currency、calculation_method
回答中明确口径
不同口径不得直接比较

7.4 权限和合规风险

研报可能存在权限限制：

- 内部报告；

- 付费报告；
- 未公开材料；
- 客户敏感信息；
- 合规隔离要求。

控制方法：

文档级权限控制
用户级 ACL
检索前先做权限过滤
日志审计
敏感内容脱敏

7.5 投资建议合规风险

模型可能把研报观点包装成直接投资建议，例如：

“建议买入某股票。”

这在金融场景有合规风险。

控制方法：

区分“研报原文观点”和“系统建议”
输出免责声明
禁止生成个性化投资建议
回答中说明资料来源和日期

7.6 表格和 PDF 解析风险

研报大量信息在表格、图表、脚注里。普通 PDF 解析可能导致：

1. 表格错行；
2. 单位丢失；
3. 页眉页脚混入正文；
4. 图表标题和数据分离；
5. 脚注缺失。

控制方法：

使用高质量 PDF parser
表格单独结构化
图表标题、单位、来源一起保存
重要财务表进入结构化数据库

7.7 多来源冲突风险

不同券商对同一公司可能观点不同。

例如：

- A 券商维持买入；
- B 券商下调评级；
- C 券商认为盈利承压。

RAG 不能把它们合成一个“统一结论”而不说明来源差异。

控制方法：

按来源分组回答
显示分歧
不要强行归纳成单一确定结论

8. 如何设计一个“研报问答 RAG 系统”

会按“数据层 → 索引层 → 检索层 → 生成层 → 评估与治理层”来设计。

8.1 目标定义

系统目标不是简单聊天，而是：

让用户能够基于研报资料进行可追溯、可审计、低幻觉的问答、总结、比较和观点提取。

核心问题类型包括：

1. 个股研报问答；
2. 行业观点总结；

3. 多篇研报横向比较；
4. 评级和目标价变化追踪；
5. 盈利预测和估值假设提取；
6. 风险提示归纳；
7. 原文引用溯源。

8.2 数据接入层：Loading

数据来源：

```
PDF 研报
Word / PPT 材料
HTML 网页研报
券商研究数据库
公司公告
财务数据库
内部评论和会议纪要
```

LlamaIndex 中 Loading 的目标是把文件、PDF、网站、数据库或 API 等数据加载进工作流。

会将不同来源统一转换为：

```
Document - text - metadata - source info
```

同时建立文档主表：

```
doc_id
title
broker
analyst
publish_date
industry
stock_code
company_name
report_type
permission_level
file_path
```

8.3 文档解析与清洗层

重点处理：

1. PDF 正文抽取；
2. 页眉页脚删除；
3. 目录、免责声明识别；
4. 表格抽取；
5. 图表标题、单位、来源保留；
6. 页码映射；
7. 标题层级识别；
8. 风险提示、投资要点、盈利预测等 section 识别。

研报解析不能只拿纯文本，因为大量关键信息在表格和图表中。

8.4 Chunk 与 Node 设计

LlamaIndex 中 Node 是 RAG 检索的原子单位，代表 Document 的一个 chunk。

采用“结构感知 chunking”，而不是简单每 500 字切一刀。

正文 chunk

按标题层级 + 段落切分
chunk_size: 500-1000 中文字
overlap: 50-150 字

投资要点

每个 bullet 可单独作为 node
同时保留整个“投资要点”section 的 summary node

表格

表格单独保存为结构化 table node
保留表名、单位、行列、页码、来源

风险提示

风险提示单独切分

metadata 标记 `section_type = risk`

每个 Node 的 metadata:

```
node_id
doc_id
title
broker
analyst
publish_date
industry
stock_code
company_name
section_title
section_type
page_number
chunk_index
permission_level
```

8.5 索引层: Indexing

主索引采用:

VectorStoreIndex + Keyword Index + Metadata Index

LlamaIndex 中 VectorStoreIndex 会把 Documents 切成 Nodes, 并为每个 node 创建 vector embeddings。

为什么不只用向量索引?

因为金融问题中有大量精确实体:

股票代码
公司名
指标名
报告日期
券商名称
分析师名称

评级
目标价

所以需要：

1. 向量索引：语义召回；
2. 关键词索引：实体和指标精准召回；
3. metadata 索引：时间、行业、权限、报告类型过滤；
4. 结构化数据库：财务指标和评级目标价等强事实。

8.6 存储层：Storing

LlamaIndex 文档说明，索引默认只存在内存里，通常需要持久化以避免重复 re-index。

分层存储：

对象存储：PDF 原文
关系型数据库：文档元数据、用户权限、结构化指标
向量数据库：chunk embedding
搜索引擎：BM25 / 关键词索引
缓存：高频 query 和热门报告摘要

技术选型示例：

PostgreSQL：文档主表、元数据、权限
Milvus / Qdrant / Chroma：向量库
Elasticsearch / OpenSearch：关键词检索
对象存储：PDF 原文
Redis：缓存

8.7 查询理解层

用户问题进入系统后，先做 query understanding：

识别问题类型：

- 个股问答
- 行业总结
- 多报告比较
- 指标查询

- 风险提示
- 评级/目标价
- 时间变化

识别实体：

- 公司名
- 股票代码
- 行业
- 时间范围
- 券商
- 指标
- 报告类型

例如：

“最近三个月券商对宁德时代盈利压力怎么看？”

解析为：

```
{  
  "company": "宁德时代",  
  "time_range": "最近三个月",  
  "intent": "观点归纳",  
  "topic": "盈利压力",  
  "source_type": "研报"  
}
```

8.8 检索层：Hybrid Retrieval

查询流程：

1. 权限过滤
2. 时间过滤
3. 公司/行业/券商 **metadata** 过滤
4. **BM25** 关键词召回
5. **Embedding** 语义召回
6. 合并候选结果
7. **Rerank**
8. 选择最终上下文

LlamaIndex 文档提到，`retriever` 定义如何从 `index` 中高效检索相关上下文，检索策略对相关性和效率很关键。

8.9 Rerank 层

引入 reranker，对候选 nodes 做重新排序。

排序特征包括：

语义相关性
关键词命中
实体一致性
时间新鲜度
section_type 权重
券商/报告权威性
是否包含明确答案
是否包含表格或关键数据

例如用户问“风险”，则提高：

```
section_type = risk  
section_title 包含“风险提示”
```

用户问“盈利预测”，则提高：

```
section_type = forecast  
包含 EPS、净利润、收入、毛利率等字段
```

8.10 生成层：Response Synthesis

LlamaIndex 中 Response Synthesizer 会把用户 query、检索到的文本块和 prompt 结合，由 LLM 生成回答。

要求模型输出结构化答案：

结论摘要
分点说明
证据引用
不同来源分歧
风险提示
资料日期
不确定性说明

示例 prompt 规则：

你只能基于提供的研报片段回答。
不得编造未提供的数据、评级、目标价。
每个关键结论必须附来源。
如果资料不足，回答“当前资料不足以判断”。
如果不同来源观点冲突，必须分别列示。

8.11 引用溯源层

每个答案后附引用：

[1] 报告标题，券商，分析师，发布日期，第 X 页，**section[2]** 报告标题，券商，发布日期，第 Y 页

前端支持：

- 1. 点击引用跳转原文 PDF 页码；
- 2. 展示原文高亮片段；
- 3. 查看上下文前后段；
- 4. 下载或打开原报告。

这依赖 ingestion 阶段保存好 page_number、section_title、chunk_id、source_path 等 metadata。

8.12 结构化数据增强

对以下强事实，不建议只依赖文本生成：

评级
目标价
EPS
PE
收入
净利润
毛利率
ROE

估值假设
盈利预测表

这些应当通过抽取进入结构化数据库。

当用户问：

“过去半年哪些券商上调了宁德时代目标价？”

系统应该查结构化表，而不是让 LLM 从一堆文本里自由总结。

8.13 评估体系

LlamaIndex 把 Evaluation 列为 RAG 的关键阶段，用于衡量回答的准确性、忠实性和速度。

我会设计四类评估指标：

检索评估

Recall@K
MRR
NDCG
命中正确报告比例
命中正确 section 比例

回答评估

faithfulness: 是否忠实原文
answer relevance: 是否回答问题
citation accuracy: 引用是否支持结论
completeness: 是否遗漏关键点

金融专项评估

数字是否准确
时间是否准确
口径是否一致
评级/目标价是否编造
是否混用不同报告观点

工程评估

响应延迟
token 成本
失败率
无答案率
权限拦截准确率

8.14 安全与合规设计

必须有：

用户权限控制
文档权限控制
敏感信息脱敏
日志审计
投资建议边界
回答免责声明
不可回答问题策略

例如当用户问：

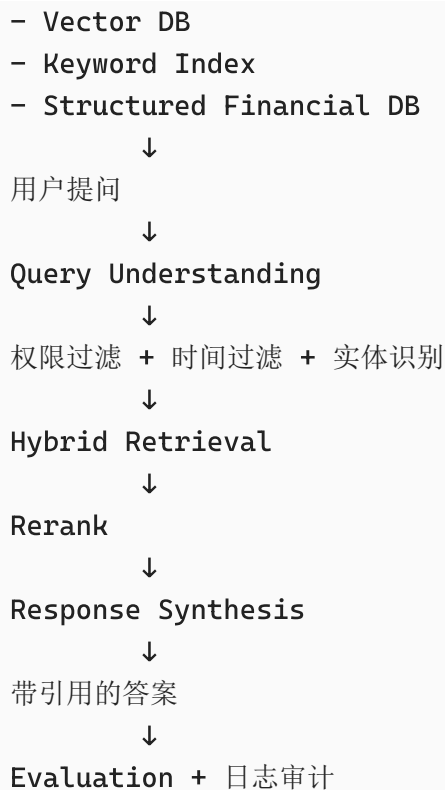
“我现在应该买入这只股票吗？”

系统应回答：

我可以基于研报资料整理不同机构观点、风险提示和估值假设，但不能提供个性化投资建议。

8.15 最终架构图

研报 PDF / 数据库 / 网页 / 内部文档
↓
数据加载 Loading
↓
解析清洗：正文、表格、图表、页码、标题层级
↓
Chunking + Metadata + Embedding
↓
存储：
- 原文对象存储
- Metadata DB



总结

这 8 个问题可以归结为一个核心判断：

RAG 不是简单地“把文档丢给大模型”，而是一套围绕数据加载、切分、索引、检索、重排、生成、引用和评估的工程系统。

在金融研报场景里，一个合格的 RAG 系统必须做到：

1. 知识可更新；
2. 检索可解释；
3. 答案有引用；
4. 数据有口径；
5. 权限可控制；
6. 观点不混淆；
7. 风险可提示；
8. 输出可审计。